

Derin Öğrenme Projesi - Sunum Notları ve Kod Açıklamaları

Merhaba Hocam/Arkadaşlar, bugün sizlere MNIST veri seti üzerinde geliştirdiğimiz derin öğrenme projemizi sunacağız. Projemizi iki ana aşamaya böldük: İlk kısımda Autoencoder'lar ile görüntüyü sıkıştırıp yeniden oluşturmayı denedik, ikinci kısımda ise LSTM ile görüntüleri ardışık veri (sequence) olarak işleyip sınıflandırma yaptık. Şimdi kodları adım adım açıklamak istiyorum.

BÖLÜM 1: AUTOENCODERS (autoencoders.ipynb)

Bu bölümde amacımız, 28x28 boyutundaki görüntüleri alıp çok daha küçük bir boyuta (64 boyutlu latent space) sıkıştırmak ve ardından bu sıkıştırılmış veriden orijinal görüntüyü geri oluşturmaktır. İki farklı mimari denedik: Biri düz sinir ağı (Fully Connected), diğeri ise evrişimli (Convolutional).

1. Veri Yükleme (Data Loading)

```
def load_idx_images(path):
    with open(path, 'rb') as f:
        _, n, rows, cols = struct.unpack('>IIII', f.read(16))
        data = np.frombuffer(f.read(), dtype=np.uint8)
    return data.reshape(n, 1, rows, cols).astype(np.float32) / 255.0
```

Ne Yaptık? MNIST veri seti .idx formatında geldiği için Python'un struct kütüphanesiyle bu binary dosyaları doğrudan okuduk. **Neden Yaptık?** Veriyi modele vermeden önce 0-255 arasındaki piksel değerlerini / 255.0 yaparak [0, 1] aralığına normalize ettik. Çünkü sinir ağları küçük ve normalize edilmiş değerlerle çok daha hızlı ve stabil öğrenir.

2. Task 1: Fully Connected (FC) Auto-Encoder

```
class FCEncoder(nn.Module):
    def __init__(self, input_dim=784, hidden_dim=256, latent_dim=64):
        # ...
        self.encoder = nn.Sequential(
            nn.Linear(input_dim, hidden_dim), # 784 -> 256
            nn.ReLU(),
            nn.Linear(hidden_dim, latent_dim), # 256 -> 64
            nn.ReLU()
        )
```

Ne Yaptık? Klasik bir Encoder yazdık. Görüntü (28x28 = 784 piksel) olarak giriyor, 256 nöronluk bir gizli katmandan geçip en sonda 64 nöronluk bir darboğaza (bottleneck) sıkışıyor. **Neden Yaptık?** Görüntüdeki en önemli özellikler (features) sadece 64 boyutta tutabilmesi için bu mimariyi seçtik. Aktivasyon fonksiyonu olarak ReLU kullandık çünkü ölöl nöron problemini (vanishing gradient) engellemede oldukça başarılı.

```
class FCDecoder(nn.Module):
    # ...
    self.decoder = nn.Sequential(
```

```
nn.Linear(latent_dim, hidden_dim), # 64 -> 256
nn.ReLU(),
nn.Linear(hidden_dim, output_dim), # 256 -> 784
nn.Sigmoid() # Çıkış [0, 1] aralığında
)
```

Ne Yaptık? Sıkıştırılmış 64 boyutlu veriyi alıp tekrar 784 piksele (orijinal görüntü boyutuna) çeviren bir Decoder yazdık. **Neden Sigmoid Kullandık?** En sonda Sigmoid kullanmamızın sebebi, görüntüyü okurken [0, 1] aralığına normalize etmiştik. Çıkış piksellerinin de bu aralıkta olmasını garantilemek istedik.

3. Task 2: Convolutional Auto-Encoder

```
class ConvEncoder(nn.Module):
    def __init__(self, latent_dim=64):
        # ...
        self.conv_layers = nn.Sequential(
            nn.Conv2d(1, 16, kernel_size=3, stride=2, padding=1), # 28x28 -> 14x14
            nn.ReLU(),
            nn.Conv2d(16, 32, kernel_size=3, stride=2, padding=1), # 14x14 -> 7x7
            nn.ReLU()
        )
        self.fc = nn.Linear(32 * 7 * 7, latent_dim)
```

Ne Yaptık? FC ağırlar görüntünün 2 boyutlu yapısını (komşu pikseller arasındaki ilişkiyi) bozar. Biz de Conv2d kullanarak daha gelişmiş bir Encoder yazdık. İki tane evrişim katmanı ile görüntünün boyutunu 7x7'ye kadar düşürüp ardından yine 64 boyuta çektik. **Neden Yaptık?** Convolutional katmanlar, görüntü işlemede her zaman düz ağırlardan daha başarılıdır. Bakalım sıkıştırma kalitemizi gerçekten artıracak mı diye bu yapıyı kurduk.

4. Eğitim Döngüsü (Training Loop)

```
optimizer = optim.Adam(model.parameters(), lr=lr)
criterion = nn.MSELoss()

for (images,) in dataloader:
    x_hat = model(images)
    loss = criterion(x_hat, images)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

Ne Yaptık? Modeli eğitmek için Adam optimizer'ı kullandık. Kayıp fonksiyonu (loss function) olarak MSE (Mean Squared Error) seçtik. Görüntüyü modele verip (images), modelin oluşturduğu kopyayı (x_hat) aldık ve bu ikisi arasındaki piksel piksel farka bakarak modeli eğittik.

5. t-SNE Görselleştirmesi

```
tsne = TSNE(n_components=2, random_state=42, perplexity=30)
```

```
coords = tsne.fit_transform(embeddings)
```

Ne Yaptık? Modelin 64 boyutlu darboğazında (latent space) oluşan o kodları (embeddings) aldık ve t-SNE kullanarak 2 boyuta düşürüp çizdirdik. **Neden Yaptık?** Modelin rakamları kendi kendine (unsupervised) gruplayıp gruplayamadığını, yani gerçekten rakamların özelliklerini öğrenip öğrenmediğini görmek istedik.

BÖLÜM 2: LSTM İLE SINIFLANDIRMA (`lstm.ipynb`)

Hocam, projemizin ikinci kısmında Autoencoder'lardan çıkıp sınıflandırma problemine yöneldik. Görüntüleri CNN ile değil, sanki bir zaman serisiymiş (zamanla akan veriler) gibi değerlendirip LSTM kullandık.

1. Model Mimarisi: Neden LSTM?

```
class LSTMClassifier(nn.Module):
    def __init__(self, input_size=28, hidden_size=128, num_classes=10):
        super().__init__()
        self.lstm = nn.LSTM(input_size, hidden_size, batch_first=True)
        self.fc = nn.Linear(hidden_size, num_classes)
```

Ne Yaptık? 28x28 boyutundaki bir görüntüyü alıyoruz. Ancak bunu tek bir kare gibi değil, yukarıdan aşağıya akan 28 satır (her satır 28 piksel genişliğinde) olarak LSTM modelimize veriyoruz. Model her satırı bir "time step" (zaman adımı) olarak okuyor. Son satırı da okuduktan sonra elde ettiği `hidden_size=128` boyutundaki bilgiyi bir Fully Connected katmana verip 10 farklı rakam sınıfı için tahmin üretiyor.

2. Eğitim Parametreleri

```
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
```

Neden Bu Fonksiyonları Seçtik? Çok sınıflı (multi-class) sınıflandırma yaptığımız için `CrossEntropyLoss` kullandık. Optimizasyon için ise yine en hızlı ve stabil sonuçları veren `Adam` algoritmasını tercih ettik.

3. Hyperparameter Tuning (Parametre Optimizasyonu) - Ekstra Çalışmamız

```
# Kendi kurduğumuz Hyperparameter Tuning Döngüsü
hidden_sizes = [64, 128]
learning_rates = [1e-3, 5e-4]
batch_sizes = [64, 128]

for hs in hidden_sizes:
    for lr in learning_rates:
        for bs in batch_sizes:
            acc = train_and_evaluate(hidden_size=hs, lr=lr, batch_size=bs,
                                     epochs=10)
```

Ne Yaptık? "Hangi ayarlar en iyi sonucu verir?" diye tahmin yürütmek yerine, bilimsel bir yaklaşım sergileyerek bir Grid Search (Izgara Taraması) tasarladık. LSTM kapasitesini (Hidden Size), öğrenme hızını (Learning Rate) ve veri yığın boyutunu (Batch Size) değiştirerek toplam 8 farklı modeli 10'ar epoch boyunca

eğittik. **Sonuç:** Model varsayılan ayarlarıyla %98.24 doğruluk verirken, Batch Size'ı 64'e düşürdüğümüzde modelin ağırlıklarını daha sık güncellediğini gözlemledik ve doğruluğu **%98.95'e** çıkardık.

4. t-SNE ve K-Means Kümeleme (Clustering)

```
# Modeli test setiyle değerlendirdikten sonra
kmeans = KMeans(n_clusters=10, random_state=42, n_init=10)
cluster_labels = kmeans.fit_predict(coords_2d)
centroids = kmeans.cluster_centers_
```

Ne Yaptık? LSTM ağının son zaman adımından çıkan 128 boyutlu hafızayı (hidden state) aldık. Önce t-SNE ile 2 boyuta düşürüp haritasını çıkardık, ardından K-Means ile 10 farklı kümeye böldük ve merkezlerini işaretledik. **Neden Yaptık?** Amacımız modelin sadece test sonuçlarında değil, hafıza düzeyinde de sayıları ne kadar iyi birbirinden ayırdığını (embedding clustering) görsel olarak kanıtlamaktı.

(Sunumu burada sonlandırıp soruları alabiliriz.)